

Standalone Coupon Service Integration

Project Approach

1. Executive Summary & Core Approach

To implement robust coupon support without altering or destabilizing the foundational code of the existing Pizza Ordering platform, the recommended implementation is a standalone, decoupled Coupon Service.

The system relies on an authoritative double-validation pattern split into two operational flows:

Interactive Preview (Frontend): As the customer selects pizzas and enters a coupon code, the React application sends a debounced request to the Coupon Service through Azure API Management (APIM). This gives instant user feedback and updates the pricing layout dynamically.

Authoritative Checkout (Backend): When the user clicks Submit Order, the request goes exclusively to the existing Order API. The Order API communicates directly with the Coupon Service through a secure backend channel to validate the coupon and calculate the true final price before executing payments. This eliminates client-side tampering.

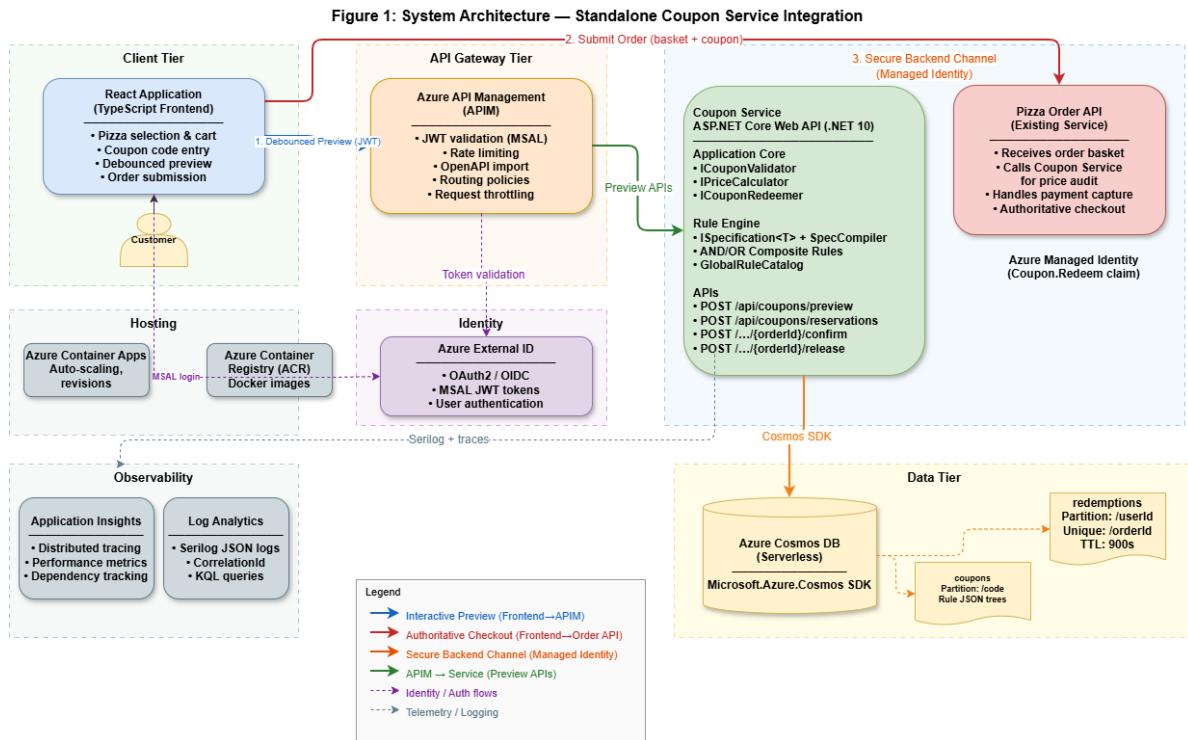
2. Technology Stack

Layer	Technology	Why This Stack
Backend	ASP.NET Core Web API, .NET 10	Cross-platform, high performance, native Azure integration, long-term support
Rule Engine	Composite Specification Pattern (ISpecification<T>)	Data-driven rules without code changes, extensible via Open/Closed Principle
Data Store	Azure Cosmos DB (Serverless), Microsoft.Azure.Cosmos SDK	Sub-millisecond reads, auto-scaling, TTL for abandoned checkouts, serverless = pay per request
Frontend	React TypeScript	Component-based UI, type safety, large ecosystem, easy MSAL integration
API Gateway	Azure API Management (APIM)	Built-in JWT validation, rate limiting, OpenAPI support, unified entry point
Identity	Azure External ID + MSAL JWTs, Azure Managed Identity	No custom auth code, passwordless service-to-service, Azure-native zero-trust
Hosting	Azure Container Apps, Azure Container Registry (ACR), Azure Static Web Apps	Serverless containers with auto-scaling, built-in CDN for frontend, no VM management
CI/CD	Azure DevOps YAML Pipeline + Bicep	Pipeline-as-code, declarative infrastructure, single tool for source + CI + deploy
Logging	Serilog → Application Insights + Log Analytics	Structured JSON logs, distributed tracing, KQL queries for troubleshooting
Testing	xUnit + Reqnroll (BDD) + FluentAssertions	Industry-standard .NET test runner, Gherkin scenarios readable by non-developers
Secrets	Azure Key Vault	Centralized secret management, Managed Identity access, no secrets in code or config

The implementation will be delivered in small, verifiable steps so each assignment requirement is covered directly in the solution design, source repository, deployment pipeline, and test suite.

- 1. Establish Azure DevOps repository structure:** create a single repository containing the .NET 10 Coupon Service, React TypeScript frontend, BDD test project, Bicep infrastructure templates, and Azure DevOps YAML pipeline. This satisfies the required source code and repository deliverable.
- 2. Build the standalone backend service:** implement an ASP.NET Core Web API on .NET 10 with clean interfaces for coupon validation, price calculation, and redemption lifecycle management. The service will expose preview APIs for frontend feedback and backend-only mutation APIs for reserve, confirm, and release operations.
- 3. Add coupon support to the pizza ordering flow:** the React frontend will allow customers to select pizzas, enter a coupon code, see validation feedback, submit the order, and view subtotal, discount, and final total. The frontend calculation is treated as preview only.
- 4. Protect the final order price:** on order submission, the existing Order API will send the full basket to the Coupon Service and use the service-calculated audited total before payment capture. This ensures coupon discounts affect the final price while preventing client-side tampering.
- 5. Expose APIs through Azure APIM:** public preview endpoints and backend integration endpoints will be represented in OpenAPI and imported into APIM during deployment. APIM will enforce token validation, routing, rate limits, and policy-based access control.
- 6. Secure all API paths:** users authenticate through Azure External ID and MSAL-issued JWTs. APIM validates public requests, while service-to-service calls from the Order API to mutation endpoints use Azure Managed Identity and application-role checks.
- 7. Implement automated testing (two-stage strategy):** Stage 1 — xUnit service-level tests for PricingService, CouponService, and Rule Engine specifications run during CI. Stage 4 — Reqnroll BDD tests execute real HTTP calls through APIM → API → Cosmos DB on the live deployed environment.
- 8. Implement structured logging and monitoring:** use Serilog with structured JSON output and enrich each request with CorrelationId, UserId, CouponCode, and OrderId. Logs and traces will flow to Application Insights and Log Analytics.
- 9. Automate deployment from scratch:** Bicep templates will provision APIM, Azure Container Apps, ACR, Cosmos DB containers, Static Web Apps, Managed Identities, Application Insights, and Log Analytics. The Azure DevOps YAML pipeline will run build, test, provision, deploy, and verification stages without manual Azure portal steps.
- 10. Document architecture, deployment, authentication, and assumptions:** the repository will include documentation explaining architecture choices, deployment steps, authentication approach, and assumptions requiring client confirmation.

3. System Architecture



The architecture establishes clear tier boundaries:

Client Tier: React Application (TypeScript) pizza selection, cart management, coupon code entry with debounced preview, order submission. Communicates with APIM for preview and with Order API for authoritative checkout.

API Gateway Tier: Azure API Management (APIM) — JWT validation via MSAL, rate limiting, OpenAPI-based routing, request throttling. All public-facing requests pass through APIM.

Identity Tier: Azure External ID — OAuth2/OIDC authentication, MSAL JWT token issuance. APIM validates tokens; React app acquires tokens via MSAL library.

Backend Service Tier: Coupon Service (ASP.NET Core Web API, .NET 10) Application Core: ICouponValidator, IPriceCalculator, ICouponRedeemer. Rule Engine: ISpecification<T>, SpecCompiler, AND/OR composite rules, GlobalRuleCatalog. APIs: Preview, Reserve, Confirm, Release endpoints. Pizza Order API (Existing Service) calls Coupon Service via Azure Managed Identity with Coupon.Redeem claim.

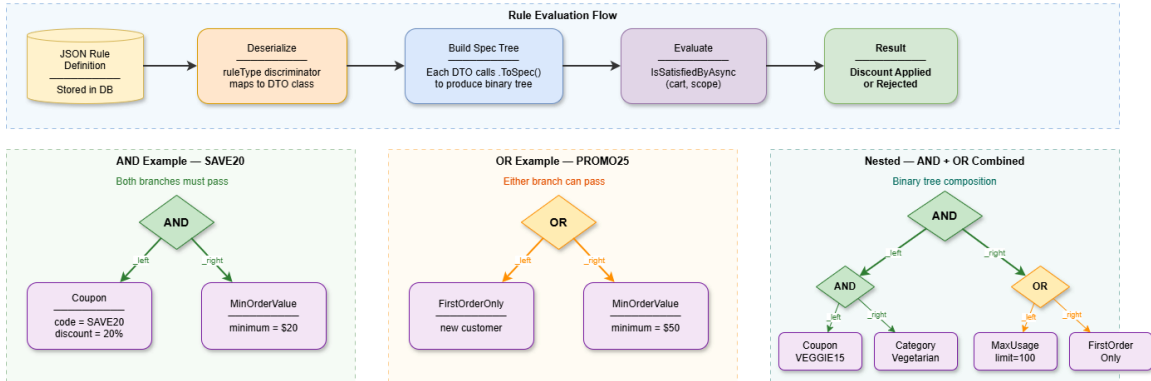
Data Tier: Azure Cosmos DB (Serverless) via Microsoft.Azure.Cosmos SDK — coupons container (partitioned by /code for fast point reads), redemptions container (partitioned by /userid for per-user usage caps, unique key /orderid for idempotency).

Observability Tier: Application Insights — distributed tracing, performance metrics. Log Analytics — Serilog structured JSON logs, CorrelationId enrichment, KQL queries.

Hosting Tier: Azure Container Apps — auto-scaling, revision management. Azure Container Registry (ACR) Docker image storage.

4. Rule Engine — Composite Specification with AND/OR

Figure 2: Rule Engine — AND / OR Composite Specification Pattern



The Rule Engine uses the Composite Specification Pattern with a binary tree structure to evaluate coupon eligibility. Rules are stored as JSON documents in the database and compiled at runtime into executable specification trees.

DTO Layer & Polymorphic Deserialization

The abstract RuleDtoBase class defines ToSpec() — each DTO subclass produces its own specification:

ruleType (JSON)	DTO Class	ToSpec() Output
"And"	CompositeRuleDto	AndSpec<Cart> (nested binary tree)
"Or"	CompositeRuleDto	OrSpec<Cart> (nested binary tree)
"Coupon"	CouponRuleDto	CouponSpec
"MinimumOrderValue"	MinOrderValueRuleDto	MinimumOrderValueSpec
"MaxUsageLimit"	MaxUsageLimitRuleDto	MaxUsageLimitSpec
"ApplicableCategory"	ApplicableCategoryRuleDto	ApplicableCategorySpec

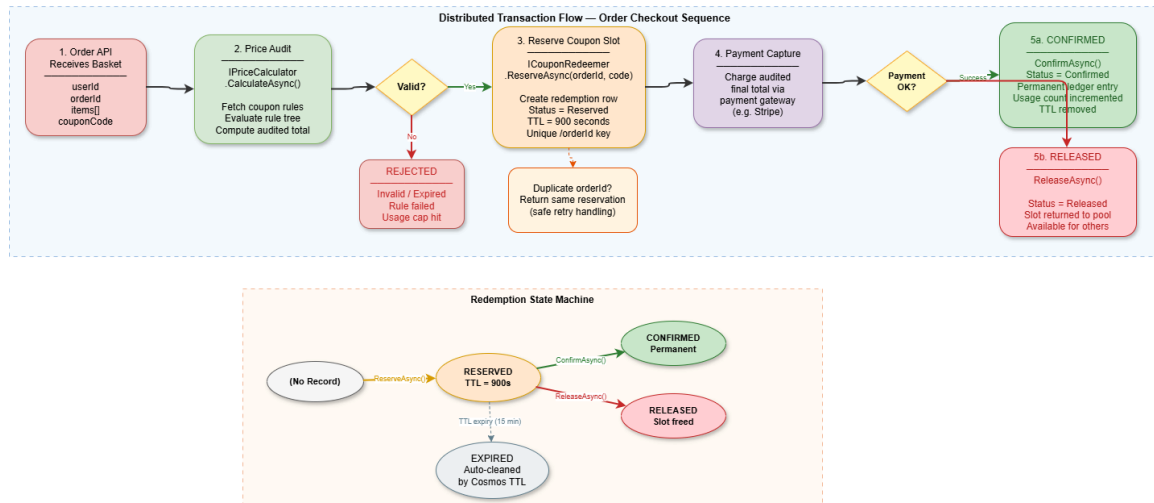
CompositeRuleDto is a single DTO class for both AND and OR. It holds Rules: IReadOnlyList<RuleDtoBase> and uses the ruleType discriminator to determine whether to produce AndSpec or OrSpec nodes.

RuleDtoConverter (JsonConverter<RuleDtoBase>) reads the "ruleType" discriminator from JSON and routes to the correct DTO subclass.

Key: A coupon is NOT a separate entity — it is a Rule whose JsonDefinition contains a "Coupon" leaf node. Campaign managers configure rules as JSON data — no code changes needed.

5. Transaction Management — Reserve-Confirm-Release

Figure 3: Transaction Management — Reserve-Confirm-Release Pattern



To handle distributed environments without database locks across independent systems, the Coupon Service uses an asynchronous two-phase state machine.

6. Data-Driven Rule Composition (AND / OR Conditions)

Coupon eligibility is represented as a composable JSON rule tree. The validation engine evaluates leaf rules while composite nodes (AND / OR) define how child rules are combined. Multi-child JSON arrays are compiled into nested binary trees at runtime.

AND Rule — All Conditions Must Pass

Example: `{ "ruleType": "And", "rules": [ { "ruleType": "Coupon", "code": "SAVE20", "discountPercentage": 20 }, { "ruleType": "MinimumOrderValue", "minimumValue": 20.00 } ] }`

Compiled binary tree: `AndSpec(_left: CouponSpec, _right: MinimumOrderValueSpec)`. Both branches must pass.

OR Rule — At Least One Condition Must Pass

Example: `{ "ruleType": "Or", "rules": [ { "ruleType": "FirstOrderOnly" }, { "ruleType": "MinimumOrderValue", "minimumValue": 50.00 } ] }`

Compiled binary tree: `OrSpec(_left: FirstOrderOnlySpec, _right: MinimumOrderValueSpec)`. Either branch can pass.

Nested AND + OR — Complex Campaign Logic

`AND(AND(Coupon, Category), OR(MaxUsage, FirstOrder))` — rules as data, no code changes for new campaigns.

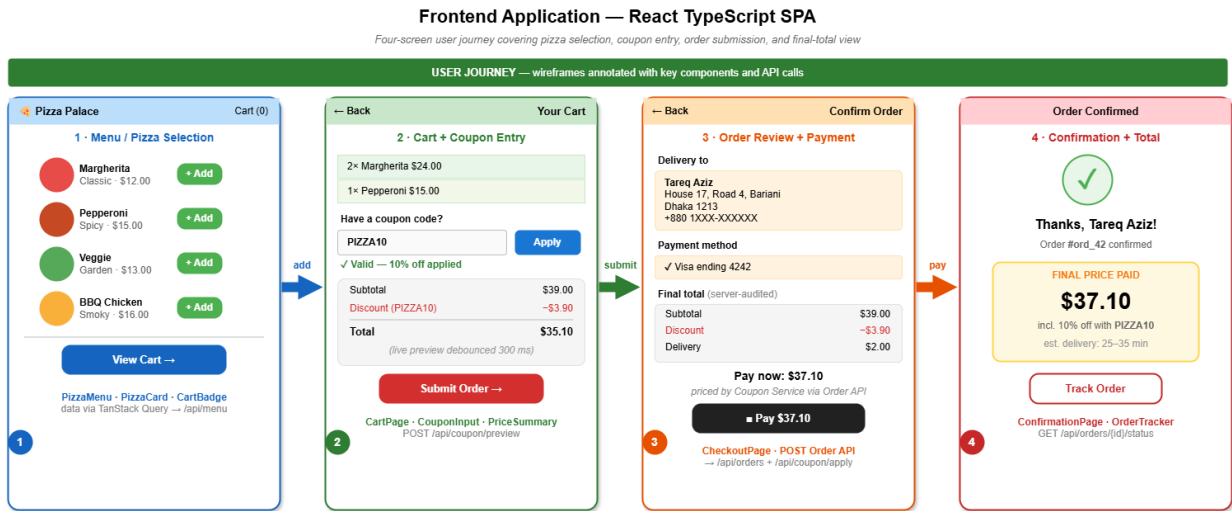
Supported Leaf Rules

Rule	Parameters	Description
Coupon	code, discountPercentage	Matches coupon code (case-insensitive), sets discount %
MinimumOrderValue	minimumValue	Cart subtotal must meet or exceed threshold
MaxUsageLimit	maxUsage	Global usage count must be below limit
ApplicableCategory	category	All cart items must belong to specified category

FirstOrderOnly	—	Customer must be a first-time buyer
WeekendOnly	—	Order must be placed on Saturday or Sunday

7. Frontend Application — React TypeScript SPA

The customer-facing application is a React 18 + TypeScript single-page app built with Vite and Material UI. It delivers the four user actions called out in the brief — select pizzas, enter a coupon code, submit the order, and view the calculated total — and intentionally holds no business rules. Every price the user sees is a hint until the Order API re-prices server-side.



8. Infrastructure & Persistence Layer

Cosmos DB Container Design

coupons container: Partition Key: /code — fast point reads during validation. Contains JSON rule trees defining coupon eligibility.

redemptions container: Partition Key: /userId — efficient per-user usage cap queries. Unique Key: /orderId — idempotency against network retries. TTL: 900 seconds on Reserved entries — self-cleaning abandoned checkouts.

The Infrastructure layer uses the official Microsoft.Azure.Cosmos SDK. The Application Core accesses storage only through ICouponRepository and IRedemptionRepository, keeping Cosmos SDK types outside business logic.

9. Security, Observability & Authentication

9.1 Perimeter Authentication Architecture

Layer	Mechanism
User Authentication	Azure External ID — OAuth2 JWT via MSAL
Gateway Filtering	APIM enforces signature, expiry, audience verification
Service Trust Boundaries	Azure Managed Identity — mutation paths require Coupon.Redeem application claim

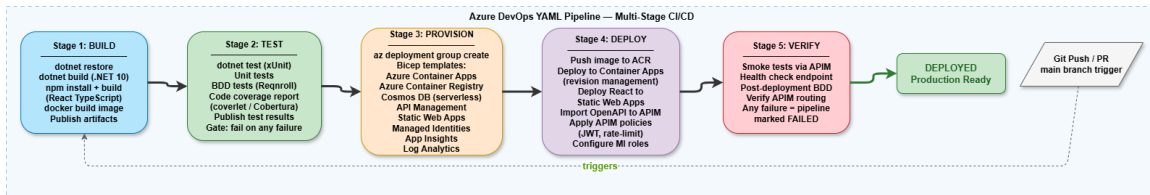
9.2 Logging & Operational Monitoring

Serilog structured JSON logs emitted to Azure Application Insights and Log Analytics. Inbound middleware enriches every log entry with:

- CorrelationId — end-to-end request tracing
- UserId — customer identification
- CouponCode — coupon being evaluated
- OrderId — order context

10. DevOps / Deployment Pipeline

Figure 4: DevOps / Deployment — Azure DevOps Multi-Stage YAML Pipeline



A single multi-stage Azure DevOps YAML pipeline automates the entire build-test-provision-deploy-verify lifecycle. No manual Azure portal steps required.

11. Testing Strategy

Figure 5: Testing Strategy



The testing strategy is split into two stages aligned with the CI/CD pipeline to ensure both isolated correctness and live environment validation.

Stage CI: xUnit Service-Level Tests

These run during the build stage with no infrastructure dependencies. They test PricingService, CouponService, and the rule engine in isolation using mocked repositories.

PricingService Tests

- Calculate subtotal for a single item — one Margherita at \$15.00 should return subtotal \$15.00
- Calculate subtotal for multiple items — two Margherita (\$15.00) and one Pepperoni (\$18.00) should return \$48.00
- Return zero for an empty cart — no items should return \$0.00 across the board

CouponService Tests

- Valid percentage coupon — PIZZA20 on a \$30.00 cart should return \$6.00 discount
- Reject expired coupon — should return invalid with a clear reason
- Reject coupon below minimum order value — \$10.00 cart with a \$20.00 minimum should fail
- Reject coupon that exceeded usage limit — maxUsage: 100 with 100 uses should fail
- Category-restricted coupon — VEGGIE15 should only discount Vegetarian items

Rule Engine (Specification) Tests

- CouponSpec matches correct code and rejects wrong code
- MinimumOrderValueSpec passes when cart total meets threshold
- AndSpec requires both child specs to be satisfied
- OrSpec passes when at least one child spec is satisfied
- SpecCompiler correctly deserializes a composite JSON rule into a spec tree

Stage Post-Deploy: Reqroll BDD Tests (End-to-End)

Reqroll with Gherkin syntax. The tests hit the actual deployed API through APIM, so they cover the full stack end to end: APIM → API → Cosmos DB.

Gherkin Scenarios

Feature: Coupon Validation and Order Pricing

As a customer ordering pizzas

I want to apply a coupon code to my order

So that I receive the correct discount

Scenario: Apply a percentage coupon to an order

Given a cart with a "Margherita" pizza priced at \$15.00

And a valid coupon "PIZZA20" with 20% discount

When the coupon is applied to the order

Then the discount should be \$3.00

And the total should be \$12.00

Scenario: Place an order without a coupon

Given a cart with a "Margherita" (\$15.00 x 2) and a "Pepperoni" (\$18.00 x 1)

When the order is submitted without a coupon

Then the subtotal should be \$48.00

Scenario: Flat discount coupon

Given a cart with items totaling \$40.00

And a valid coupon "FLAT5" with \$5.00 flat discount

When the coupon is applied to the order

Then the discount amount should be \$5.00

And the total should be \$35.00

Scenario: Expired coupon rejected

Given the coupon "OLDCODE" has expired

When I apply coupon code "OLDCODE"

Then the response status should be 400
And the error should indicate coupon expired

Scenario: Usage limit exceeded
Given the coupon "LIMITED10" has reached its max usage
When I apply coupon code "LIMITED10"
Then the coupon should not be valid

Scenario: Minimum order value not met
Given a cart with items totaling \$9.99
And a coupon "PIZZA20" requiring minimum order of \$20.00
When the coupon is applied to the order
Then the coupon should not be valid
And the error should indicate minimum order not met

Scenario: Invalid coupon code
When I apply coupon code "DOESNOTEXIST"
Then the response status should be 404

Scenario: Composite rules (And/Or)
Given a vegetarian cart totaling \$30.00
And I am a first-time customer
When I apply coupon code "VEGGIE15"
Then the coupon should be valid
And the discount percentage should be 15

Scenario: Category-restricted coupon
Given a cart with non-vegetarian items
When I apply coupon code "VEGGIE15"
Then the coupon should not be valid

Scenario: Unauthorized access returns 401
Given I am not authenticated
When I call the coupon preview endpoint
Then the response status should be 401

Any scenario failure marks the pipeline as FAILED — preventing broken deployments from reaching production.

12. API & Frontend Scope

12.1 Backend API Surface

Endpoint	Access	Purpose
POST /api/coupons/preview	Public (via APIM, JWT auth)	Validate coupon against basket, return subtotal/discount/total
POST /api/coupons/reservations	Backend-only (Managed Identity)	Reserve coupon usage slot before payment

POST .../reservations/{orderId}/confirm	Backend-only	Permanently confirm redemption after payment
POST .../reservations/{orderId}/release	Backend-only	Release slot after failed/cancelled payment

12.2 Admin API — Coupon Management (CRUD)

A simple REST API exposed through APIM for administrators to create, read, update, and delete coupon rules. Access is restricted to users with the Coupon.Admin application role.

Endpoint	Method	Purpose
/api/admin/coupons	GET	List all coupons (supports pagination and filtering by status)
/api/admin/coupons/{id}	GET	Get a single coupon by ID with its full JSON rule definition
/api/admin/coupons	POST	Create a new coupon with code, discount type, rule tree, expiry, and usage limit
/api/admin/coupons/{id}	PUT	Update an existing coupon (modify rules, expiry, limits, or status)
/api/admin/coupons/{id}	DELETE	Soft-delete a coupon (sets status to Disabled, preserves audit history)

12.2 React Frontend Scope

- Pizza selection workflow with quantity and cart total calculation
- Coupon code entry with debounced validation feedback through APIM
- Visible calculated subtotal, discount, and final total before checkout
- Submit order workflow sending basket to Order API (backend-authoritative pricing)

13. Assumptions

1. The existing Pizza Ordering platform exposes or can accept a basket payload containing pizza items, quantities, user identity, and optional coupon code.
2. The existing Order API remains the authoritative checkout entry point and will call the Coupon Service during order submission.
3. Microsoft Entra External ID (Azure External ID) is already provisioned in the Azure tenant for customer authentication.
4. Azure subscription access, Azure DevOps repository access, and required permissions for Bicep deployments will be provided before implementation starts.
5. Payment provider integration remains owned by the existing Order API; the Coupon Service supplies validated pricing and redemption only.
6. Coupon administration data can initially be seeded through deployment scripts or controlled JSON configuration until a dedicated admin UI is requested.